SR UNIVERSITY

AI Assistant Coding
Assignment-13.4

Name of Student : V. Abhinav Batch : 41
Enrollment No. : 2303A52486

Task 1: Refactoring Data Transformation Logic

Task Description

1. **Review the legacy code that computes transformed values using an explicit loop.**
2. **Use an AI tool to suggest a more Pythonic refactoring approach.**
3. **Refactor the code using list comprehensions or helper functions while preserving the output.**

Legacy Code:

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Prompt:

Refactor this Python loop into a more Pythonic approach using list comprehension while preserving output.

Refactored Code:

Assign 13.4.py > ...

```python
# Refactor this Python loop into a more Pythonic approach using list comprehension while preserving output.
values = [2, 4, 6, 8, 10]
doubled = [v * 2 for v in values]
print(doubled)
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\A
Assign 13.4.py"
[4, 8, 12, 16, 20]
```

Explanation:

The legacy implementation used an explicit for loop and a mutable list to accumulate results. While functionally correct, this approach reflects:
Imperative programming style.
Unnecessary boilerplate code.
Reduced readability due to multiple lines for a simple transformation.

The refactored version uses list comprehension, which is:

- More declarative.
- Expressive of intent (transform every element).
- Shorter and easier to read.
- Idiomatic Python practice.

From a performance standpoint:

Time Complexity: O(n) (unchanged).
Space Complexity: O(n) (unchanged).

Slightly faster in Python due to internal optimization. Thus, the refactoring improves clarity without altering computational complexity or output behavior.

Task 2: Improving Text Processing Code Readability

Task Description:

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

Legacy Code:

```
words = ["Refactoring", "with", "AI", "improves","quality"]
message = ""
for w in words:
        message += w + " "
print(message.strip())
```

Prompt:

Suggest a more efficient method to build a sentence from a list of words in python.

Code:

```python
words = ["Refactoring", "with", "AI", "improves","quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())

# Suggest a more efficient method to build a sentence from a list of words in Python.
words = ["Refactoring", "with", "AI", "improves","quality"]
message = " ".join(words)
print(message)
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\
Assign 13.4.py"
Refactoring with AI improves quality
```

Explanation:

The legacy approach performed string concatenation inside a loop: message += w + " "

This is inefficient because Strings in Python are immutable. Each concatenation creates a new string object. Time complexity can degrade toward $O(n^2)$ for large lists. The refactored solution uses " ".join(words). This optimizes memory allocation. Linear time complexity O(n). Cleaner and more readable expression of intent. No need for .strip(). This refactoring improves both performance efficiency and code maintainability.

Task 3: Safer Access to Configuration Data

Task Description:

- **Review the legacy code that manually checks for dictionary keys.**
- **Use AI suggestions to refactor the code using safer dictionary access methods.**
- **Ensure the behavior remains the same for missing keys.**

Legacy Code:

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Prompt:

Refactor dictionary key checking using safer access methods in Python.

Code:

```python
# Task-3:
config = {"host": "localhost", "port": 8080}

if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")

# Refactor dictionary key checking using safer access methods in Python.
timeout_value = config.get("timeout", "Default timeout used")
print(timeout_value)
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\A
Assign 13.4.py"
Default timeout used
Default timeout used
```

Explanation:

The legacy code manually checked key presence if "timeout" in config

Issues:

Verbose structure. Not scalable for multiple keys. Increases cognitive load.

Risk of KeyError if incorrectly accessed. The improved version uses:

config.get("timeout", "Default timeout used")

Advantages:

Built-in safe accessor. Handles missing keys gracefully. Reduces conditional branching. More concise and expressive. This improves robustness and follows defensive programming principles.

Behavior remains identical:

If key exists → return value. If key missing → return default message.

Task 4: Refactoring Conditional Logic for Scalability

Task Description:

- **Examine the multiple if–elif conditions used to determine operations.**
- **Ask AI to suggest a cleaner, scalable alternative.**
- **Refactor the logic using mapping techniques while preserving Functionality.**

Legacy Code:

```
action = "divide"
x, y = 10, 2

if action == "add":
        result = x + y
```

```
elif action == "subtract":
        result = x - y
elif action == "multiply":
        result = x * y
elif action == "divide":
        result = x / y
else:
        result = None
        print(result)
```

Prompt:

Replace multiple if-elif conditions with a more scalable approach using mapping techniques.

Code:

```python
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    return a / b
operations = {
    "add": add,
    "subtract": subtract,
    "multiply": multiply,
    "divide": divide
}

action = "divide"
x, y = 10, 2
result = operations.get(action, lambda a, b: None)(x, y)
print(result)
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi
Assign 13.4.py"
5.0
```

Explanation:

The legacy implementation uses a chained if-elif-else structure. Problems include:

Poor scalability. Hard to extend. Violates Open-Closed Principle. Code duplication risk when logic grows. The refactored version introduces a dispatch dictionary (operation mapping):

```
operations = {
    "add": lambda a, b: a + b,
    ...
}
```

Benefits:

Constant-time lookup O(1).

Extensible design (add new operation easily).

Cleaner separation of logic.

Reduced cyclomatic complexity.

Follows functional programming style.

This design pattern improves maintainability and scalability in larger systems

Task-5: Simplifying Search Logic in Collections

Task Description:
- **Identify the explicit loop used for searching an item.**
- **Use AI assistance to refactor the logic into a more concise and readable form.**
- **Maintain the same output behavior.**

Legacy Code:

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
        if item == "eraser":
                found = True
                break
print("Item Available" if found else "Item Not Available")
```

Prompt:

Refactor the search loop to ensure it checks all items in the inventory before determining availability.

Refactored Code:

```python
# Task-5:
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
    break
print("Item Available" if found else "Item Not Available")

# Refactor the search loop to ensure it checks all items in the inventory before determining availability.
inventory = ["pen", "notebook", "eraser", "marker"]
found = any(item == "eraser" for item in inventory)
print("Item Available" if found else "Item Not Available")
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi
Assign 13.4.py"
Item Not Available
Item Available
```

Explanation:

The legacy implementation manually iterates for item in inventory:

Problems: Redundant boolean flag. Unnecessary loop control. More lines than required. Less expressive intent. Refactored version: "eraser" in inventory

Advantages:

Direct membership testing. Cleaner and more semantic.Built-in optimization. Maintains same O(n) complexity.Eliminates control flow clutter.This enhances readability and adheres to Pythonic principles.